

Architectural Tradeoffs in Low-Latency Algorithmic Trading with Predictive Signals

Abhinav Randive

February 19, 2026

1 Introduction

The introduction will broadly introduce your topic, motivate it, and describe what will be done.

2 Related Works

3 Methodology

This project begins by constructing the foundational infrastructure required for an event-driven trading system simulation. At this stage, the focus is on market data ingestion and deterministic event replay.

3.1 Market Data Acquisition

Historical market data is downloaded using Yahoo Finance API through the `yfinance` Python library. The current implementation retrieves recent S&P 500 index data at one minute intervals and stores the timestamped price and volume information in CSV format. The data preprocessing step flattens multi-index columns, standardizes column names, and converts timestamps to integer Unix time for consistent processing. This step ensures reproducibility and establishes a structured input format for the simulation pipeline.

3.2 Event Abstraction

To support an event-driven architecture, market data is abstracted into discrete events. An `Event` data structure is defined with three fields: `timestamp`, `event type` and `payload`. Event types are enumerated to allow future extension for order submission, cancellation, and execution events.

Each market update is represented as a structured event object, enabling uniform handling of data within the system. This abstraction lays the groundwork for modular system components.

3.3 Deterministic Market Replay

4 Results

An exemplary figure is shown in Figure 1. Any time a figure is used, it must have a caption and be directly referred to and discussed in the text.

5 Conclusion

References

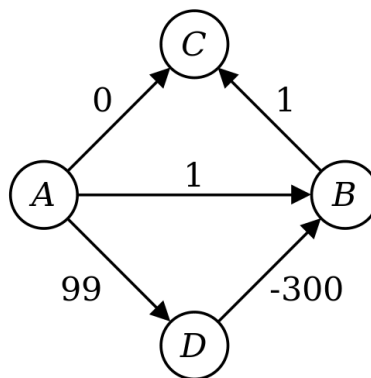


Figure 1: Dijkstra's algorithm does not work on graphs with negative edge weights, as evidenced here.